

For this assignment, we worked with CPU scheduling, once again using Python's psutil library. The program was tasked with displaying CPU usage percentage, CPU count, total memory available and memory usage. Aside from having the psutil library installed in your environment, no special instructions are needed to run the program properly, other than running the main.py file itself. For displaying CPU information, the following available methods from the psutil library were used: psutil.cpu_percent(interval=1), psutil.cpu_count(logical=True) and psutil.virtual_memory(). In code, it looks as follows:

```
def get_cpu_usage():
    """Return the current CPU usage percentage."""
    return psutil.cpu_percent(interval=1)

def get_cpu_count():
    """Return the number of logical CPU cores detected on the system.
    Giving a True value to logical parameter, we avoid only displaying physical cores
    available."""
    return psutil.cpu_count(logical=True)

def get_memory_stats():
    """
    Return the total memory, used memory, and available memory.

    Values are converted from bytes to gigabytes for readability.
    """
    BYTES_PER_GB = 1024 ** 3

    memory = psutil.virtual_memory()

    # By dividing all values obtained by memory, we display values in Gigabytes
    total = memory.total / BYTES_PER_GB
    used = memory.used / BYTES_PER_GB
    available = memory.available / BYTES_PER_GB

    return total, used, available
```

Memory information was retrieved using the virtualmemory() class, accessing specific information using the methods available for retrieving total memory (memory.total), used memory (memory.used) and available memory (memory.available). The output looks as follows:

```
===== System Resource Report =====
CPU Usage:          15.6%
CPU Cores:          8
Total Memory:       16.00 GB
Memory Usage:       7.79 GB
Memory Available:   5.27 GB
=====
```

Retrieving and displaying this information, is not only something that's available, but also something that becomes crucial for performance optimization. Monitoring system performance is extremely important for understanding how heavy the load is on the processor, something that becomes crucial on systems that are expected to maintain heavy performance (ie: Servers and applications). It also becomes extremely useful for identifying current and/or possible bottlenecks on the system, something that can limit performance and in worst case scenarios, make the system crash. This ultimately leads to developers identifying areas where applications consume excessive resources and make improvements that enhance overall performance. Improvements in memory management and multi-threading implementations can be a few of many enhancements made on the system that can reduce cpu usage and lead to better system performance.

CPU and memory usage influences in how operating systems implement their scheduling algorithms, because understanding where CPU usage is currently sitting, helps better determine what implementations need to be made to ease CPU load without quitting on essential tasks that need to be completed. Operating Systems may rely on scheduling algorithms that queue based on priority, implement threading to distribute load among different cores available, ultimately leading to a more efficient system performance. Handling system load efficiently can free the physical memory, sometimes relying on logical memory to ease the load on the actual RAM and allowing for more operations to be performed.

Overall, the implementation was quite straightforward. The only challenging part was understanding what information was expected to be retrieved and what methods the psutil library had available for retrieving that information.